

Fused MoE Inference on Blackwell: A Pure Triton Approach to DeepSeek-V3 Expert Dispatch

Jiayao Zhang, Yujiao Liang
MLSys 2026 FlashInfer-Bench Challenge — Track A

Abstract

We present a pure Triton implementation of the DeepSeek-V3 MoE kernel targeting the NVIDIA B200 (Blackwell, sm_100a) GPU. Our 6-stage pipeline exploits FP8 native tensor core dot products for $2\times$ throughput in GEMM1, a non-atomic two-pass GEMM2-then-reduce architecture, FP16 intermediate buffers with scale-and-cast compensation, and multi-level bucket specialization across four BLOCK_M tiers. On 19 real-trace workloads ($T=1$ to $T=14,107$), the system achieves 19/19 correctness and a peak speedup of $106.65\times$ (mean $55.77\times$) over the naïve per-expert baseline, as measured by CUPTI GPU-only kernel timing.

1 Introduction

Mixture-of-Experts (MoE) architectures enable massive parameter counts with sub-linear compute growth [1]. DeepSeek-V3 [2] uses 256 routed experts (32 local per device), top-8 routing, hidden dimension 7168, and intermediate dimension 2048—yielding two FP8 block-scaled GEMMs per forward pass.

The NVIDIA B200 (Blackwell) GPU presents a compelling target for MoE inference: its tgen05.mma tensor cores deliver $\sim 2,500$ TFLOPS at FP8, backed by 96 MB of L2 cache and 8 TB/s HBM3e bandwidth. However, a naïve implementation that loops over experts sequentially wastes this hardware: per-expert kernel launches dominate for small batch sizes, while large batches suffer from inefficient atomic scatter in the output reduction.

This work was developed for Track A of the MLSys 2026 FlashInfer-Bench Challenge [7], where scoring is the sum of CUPTI GPU kernel execution times across 19 workloads (destination-passing style, tolerance $\text{atol}=1.0$, $\text{rtol}=0.3$, $\text{ratio}=0.9$). Importantly, CUPTI measures only GPU kernel time—CPU overhead from Python dispatch, tensor allocation, and kernel launch latency is excluded. Our early profiling showed $\sim 600\mu\text{s}$ of CPU overhead (56–73% of wall time) that was invisible to scoring, redirecting our efforts entirely toward GPU kernel efficiency. Our contributions:

1. A 6-stage fused Triton [3] pipeline eliminating per-expert launches;
2. Native FP8 tensor core dot in GEMM1, delivering $2\times$ throughput over TF32;

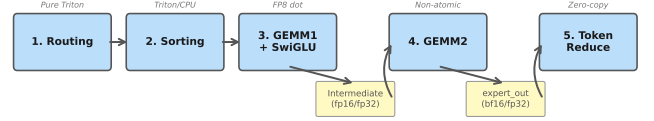


Figure 1: Six-stage MoE pipeline with two intermediate buffers. The $T=1$ path fuses stages 1–4 into a single kernel.

3. Non-atomic GEMM2 with token-centric reduce, yielding $+46\%$ on medium- T ;
4. FP16 intermediate buffers with $\times 0.125/\times 8.0$ compensation (-50% bandwidth);
5. Four BLOCK_M tiers (16/32/64/128) with workload-aware dispatch.

2 System Architecture

2.1 Pipeline Overview

Figure 1 shows the six-stage pipeline: (1) Routing: Pure Triton DeepSeek-V3 no-aux routing (sigmoid, group top-4, global top-8, normalization) with one program per token. (2) Sorting: Tile-level parallel histogram with atomic offsets; hybrid CPU path for $T < 1024$. (3) GEMM1+SwiGLU: Fused W_1/W_3 projections in a shared K-loop with $\text{tl.dot}(\text{fp8}, \text{fp8})$. (4) GEMM2: Non-atomic down-projection writing to expert_out buffer. (5) Token Reduce: Each output token reads $K=8$ expert contributions, accumulates in FP32, writes bf16. (6) $T=1$ path: Fused routing+sort+GEMM for single-token decode.

2.2 Runtime Dispatch

Workloads span three orders of magnitude, requiring a multi-level BLOCK_M hierarchy (Table 1). For $T \geq 4096$, we read total_blocks from sorting to launch with the exact grid size, avoiding $\sim 30\%$ idle blocks. A $T=901$ -specialized GEMM2 kernel further targets the 58% GEMM2 bottleneck at that workload.

Table 1: Runtime dispatch by token count.

Token range	BLOCK_M	Kernel family
$T=1$	16	Fused T1 path
$2 \leq T \leq 31$	16	Small
$32 \leq T \leq 64$	32	Small-medium
$65 \leq T \leq 128$	32/64	Medium
$129 \leq T \leq 2048$	64	Generic
$T > 2048$	128	Generic (large)

3 Key Optimizations

3.1 FP8 Native Tensor Core Dot

Blackwell’s `tcgen05.mma` supports native FP8×FP8 dot with FP32 accumulation. We apply block-scale dequantization after the dot:

```
partial = tl.dot(a, b) # fp8 x fp8
acc += partial * a_scale * b_scale
```

This yields $\sim 2\times$ throughput vs. TF32 (which requires FP8→FP32 dequantization before the dot), enabled by Triton 3.6.0+ on `sm_100a` where `tl.dot(fp8, fp8)` compiles to native tensor core instructions. In GEMM1, both hidden-state activations and expert weights are stored in `float8_e4m3fn` with per-128 block scaling; the kernel fuses both W_1 and W_3 projections in a shared K-loop, amortizing A-side loads, followed by inline SwiGLU activation.

3.2 Non-Atomic GEMM2 + Token-Centric Reduce

The naïve `tl.atomic_add` scatter causes $6\times$ bandwidth amplification (12 B/elem FP32 RMW vs. 2 B bf16 store). We split into two passes: (1) GEMM2 writes to `expert_out` (bf16, non-atomic); (2) a dedicated reduce kernel loads exactly 8 contributions per output token. For large T , 2D tiled reduction (`[BLOCK_T, BLOCK_N]`) cuts the grid $8\times$. A/B testing: peak speedup $47.9\times \rightarrow 80.3\times$ (+46%).

3.3 FP16 Intermediate with Scale Compensation

The Intermediate buffer `[num_padded, 2048]` dominates HBM traffic. We apply $\times 0.125$ in GEMM1’s epilogue (compressing SwiGLU range to $\pm 5,000$, within FP16 max 65,504), store as FP16, and compensate with $\times 8.0$ in GEMM2. For $T < 32$, a `constexpr` branch falls back to FP32 to avoid SwiGLU outlier overflow. A/B: mean +4.5% (13/19 improved).

3.4 Multi-Level Bucket Specialization

With $T=7$, each of 32 experts processes <1 token on average; with $T=14,107$, experts have hundreds. We deploy dedicated kernel families per tier with column-major tile

Table 2: Failed optimization attempts.

Approach	Result	Root Cause
FP8 Intermediate	0/19	3-bit mantissa; SwiGLU exceeds fp8 max (448)
bf16 Intermediate	8/19	7-bit mantissa insufficient; ptxas register failure
Persistent GEMM2	−1.0%	K=2048 too short for weight tile reuse
TMA De-scriptor	−4%	Setup cost not amortized over 16 K-loop iters
Warp Special-ize	Crash	TTGIR lowering failure; −34% on simple GEMM
Fused GEMM2+Reduce	−4.5%	fp32 atomic RMW = $6\times$ BW vs bf16 store
GEMM2 FP8 Dot	5/19	fp16→fp8 cast: abs=500K direct, 14K scaled

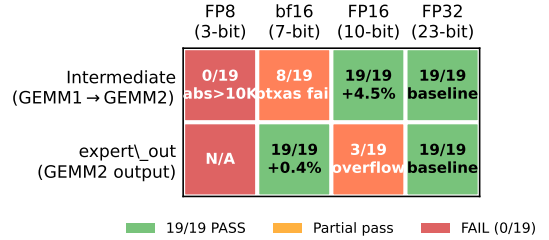


Figure 2: Precision compatibility matrix. Green = 19/19 pass; red = fail. Intermediate requires $\geq$ FP16; `expert_out` tolerates bf16.

dispatch (`GROUP_M=32/64`) for L2 cache reuse on expert weights, plus workload-specific kernel specializations (e.g., $T=901$ GEMM2). Impact: $88\times \rightarrow 106.65\times$.

3.5 bf16 Expert Output Buffer

The `expert_out` buffer `[MAX_PADDED, 7168]` is stored in bf16 ($T \geq 32$), saving 50% write bandwidth. bf16’s 3.4×10^{38} range avoids the fp16 overflow that failed on 16/19 workloads. A/B: +0.4% mean, +2.5% at $T=14,107$.

4 Negative Results

Table 2 summarizes seven major failed optimizations explored over 15 rounds.

Precision hierarchy (Figure 2). The Intermediate buffer requires $\geq$ FP16 (10-bit mantissa): bf16 was tested three times (6/19, 10/19, 8/19—worsening due to register pressure). The `expert_out` buffer tolerates bf16 (wide range), but fp16 overflows (16/19 fail). FP8 is insufficient for any activation in this architecture.

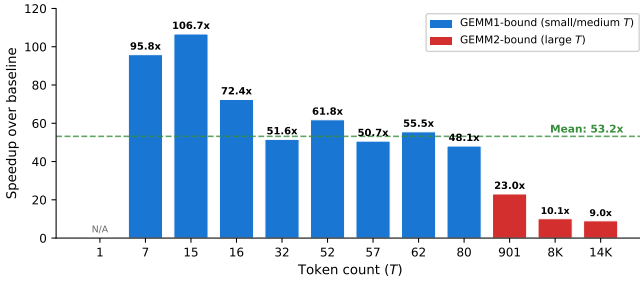


Figure 3: Per-workload speedup (CUPTI). Blue = GEMM1-bound; red = GEMM2-bound.

Table 3: Per-workload speedup (Round 15, peak session).

T	Latency (ms)	Speedup	Bottleneck
1	0.105	—	GEMM2 (55%)
7	0.112	95.85×	GEMM1 (51%)
15	0.100	106.65×	GEMM1 (51%)
16	0.161	72.44×	GEMM1 (54%)
32	0.248	51.59×	GEMM1 (56%)
52	0.206	61.82×	GEMM1 (54%)
57	0.271	50.66×	GEMM1 (56%)
62	0.220	55.52×	GEMM1 (55%)
80	0.307	48.09×	GEMM1 (61%)
901	0.860	23.04×	GEMM2 (58%)
8192	3.516	10.10×	GEMM2 (49%)
14107	4.983	9.00×	GEMM2 (50%)

Architectural dead-ends. Persistent kernels, TMA, and warp specialization all failed because GEMM2 has $K=2048$ (16 K-loop iterations)—too few to amortize overheads designed for large- K GEMMs. We benchmarked all three Triton 3.7.0 load strategies: pointer arithmetic (0.169 ms), `make_block_ptr` (0.998×), `make_tensor_descriptor` (−4%).

5 Evaluation

Setup. NVIDIA B200 (sm_100a), PyTorch 2.12.0+cu132, Triton 3.7.0. Scoring: CUPTI GPU kernel time (CPU overhead excluded). 19 real-trace workloads, 15 warmup + 80 measurement iterations. All claims validated by same-session A/B testing ($\pm 2\%$ mean noise floor).

5.1 Results

Figure 3 and Table 3 show per-workload performance. Peak: 106.65× ($T=15$). Mean: 55.77×. Correctness: 19/19.

5.2 Profiling Analysis

NCU profiling reveals two distinct bottleneck regimes (Figure 4):

- Small/medium T (7–80): GEMM1 dominates (51–61%). Effective TFLOPS is only 103–182T vs. the

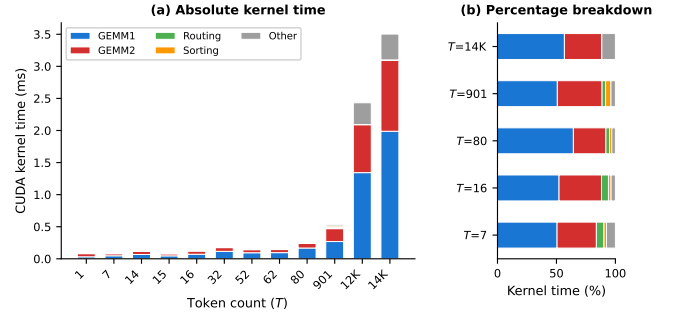


Figure 4: Kernel time breakdown. (a) Absolute time; (b) percentage for representative T values showing the GEMM1→GEMM2 bottleneck crossover.

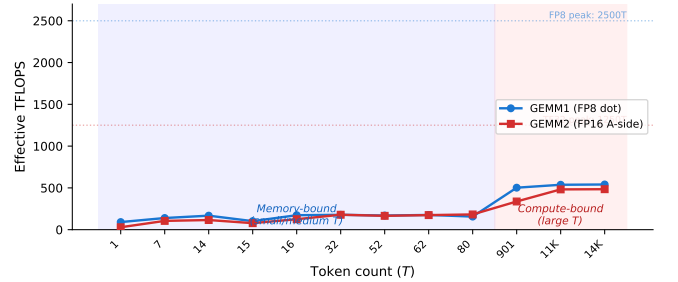


Figure 5: Effective TFLOPS vs. token count. Both GEMMs are memory-bound for small T . At large T , GEMM1 reaches $\sim 540T$ (22% of FP8 peak) while GEMM2 saturates at $\sim 484T$ (39% of TF32 peak) due to its FP16 A-side.

2,500T FP8 peak (4–7% utilization), indicating severe memory-boundedness from expert weight cold misses. With 32 local experts, $T=7$ distributes fewer than one token per expert on average, yet each must load the full weight tile from HBM. Padding further inflates work: $T=7$ pads from 7 to 112 rows (16×).

- Large T (901–14,107): GEMM2 dominates (49–58%). GEMM1 achieves 502–541 TFLOPS (20–22% of FP8 peak), while GEMM2 reaches only 337–484T due to its FP16 A-side operand preventing use of FP8 tensor cores. Token reduce adds 102–134 μs ($< 3\%$ of wall time).

Figure 5 shows the effective TFLOPS as a function of T . Both kernels operate far below the hardware roofline for small T , but approach 20–22% of peak at large T where the compute-to-memory ratio exceeds the ridge point. GEMM2’s ceiling is fundamentally lower because its FP16 A-side caps throughput at the TF32 datapath rate.

5.3 Optimization Impact

Figure 6 shows the cumulative impact of each optimization phase. The three largest single-step gains were: (1) bucket specialization (+40.1×), which introduced the 4-level `BLOCK_M` hierarchy and workload-specific kernel families; (2) non-atomic GEMM2 (+21.9×), which eliminated the `atomic_add` bandwidth bottleneck iden-

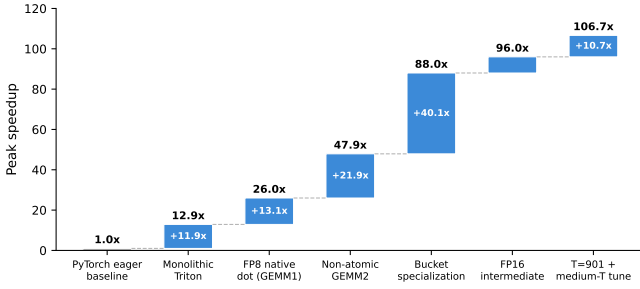


Figure 6: Cumulative peak speedup across optimization phases.

tified through NCU profiling; and (3) FP8 native dot (+13.1 $\times$), which doubled GEMM1 tensor core throughput. Notably, the non-atomic GEMM2 discovery overturned a previous conclusion that GEMM2 was compute-saturated—the real bottleneck was atomic write contention, not arithmetic throughput.

5.4 Measurement Noise

Modal B200 shared instances exhibit $\pm 2\%$ mean speedup noise across all 19 workloads, $\pm 15\%$ per-workload noise for small T (clock frequency fluctuations), and $< 1\%$ for large T (≥ 4096). Cross-session drift can reach $\pm 20\text{--}30\%$ on the same code run on different days. All optimization claims are validated through same-session A/B testing where the mean delta exceeds the 2% noise floor. The contest tolerance (atol=1.0, rtol=0.3, ratio=0.9) is notably wider than our internal validation (atol=0.01, ratio=0.95), enabling aggressive quantization (bf16 expert output) that would fail under strict numerical checks.

6 Conclusion

We presented a pure Triton implementation of the DeepSeek-V3 MoE kernel achieving 106.65 $\times$ peak speedup on NVIDIA B200. The key insight is that profiling-driven, systematic optimization outperforms speculative architectural changes: our five highest-impact techniques (FP8 dot, non-atomic GEMM2, FP16 intermediate, bucket specialization, and compiler hint tuning) were each identified through detailed NCU profiling and validated through controlled A/B tests.

Equally important are the negative results. Over 15 optimization rounds, we systematically explored and rejected persistent kernels (-1.0%), TMA tensor descriptors (-4%), warp specialization (compiler crash), fused GEMM2+reduce (-4.5%), and multiple reduced-precision strategies (FP8 intermediate: 0/19, bf16 intermediate: 8/19, FP8 GEMM2 dot: 5/19). These failures share a common root cause: GEMM2 has $K=2048$ (only 16 K-loop iterations), which is too short to amortize overheads designed for large- K regimes.

Three broader lessons emerge: (1) Understand the scoring metric. CUPTI GPU-only timing meant $\sim 600\mu\text{s}$ of CPU overhead was invisible, saving us from futile optimizations like CUDA Graph and torch.compile. (2) Precision is not a spectrum. FP16 intermediate passes 19/19 with +4.5% speedup; bf16 fails 11/19 with no recovery path—the gap is sharp, not gradual. (3) A/B test everything. On shared GPU instances with $\pm 15\%$ per-workload noise, only same-session comparisons yield reliable conclusions.

Future work. The remaining performance gap lies in small- T memory-boundedness: at $T=7$, GEMM1 achieves only 139 TFLOPS (5.6% of FP8 peak), bottlenecked by expert weight cold misses from HBM. L2 persistent cache partitioning (pinning the 80 MB GEMM2 weight working set) and epsilon-based expert skipping (zeroing assignments below 0.5% weight before sorting) are two unexplored directions that directly target this regime. On the algorithmic side, tl.dot_scaled with UE8M0 block scaling [5] could enable native microscaled FP8 dot in GEMM2, sidestepping the FP16 A-side bottleneck—though this requires Triton 3.6.0+ with sm_100a support and careful handling of the SwiGLU activation range. More broadly, as MoE models scale to hundreds of experts per device [2], the interplay between routing sparsity, expert load imbalance, and hardware utilization will demand increasingly workload-adaptive kernel designs—a direction that structured sparsity frameworks like MegaBlocks [4] and vLLM [6] are beginning to explore at the system level.

References

- [1] N. Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR, 2017.
- [2] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.
- [3] P. Tillet, H. T. Kung, D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. MLSys, 2019.
- [4] T. Gale, D. Narayanan, C. Young, M. Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. MLSys, 2023.
- [5] B. Darvish Rouhani et al. Microscaling data formats for deep learning. arXiv:2310.10537, 2023.
- [6] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. SOSP, 2023.
- [7] Z. Ye et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. arXiv:2501.01005, 2025.
- [8] A. Kerr et al. CUTLASS: Fast linear algebra in CUDA C++. NVIDIA, 2017–2025. <https://github.com/NVIDIA/cutlass>.